



INFORME DE GUÍA PRÁCTICA

1 Portada

Tema:	Implementación del Estadístico T-Student mediante Test-Driven Development (TDD) con C# y Visual Studio Code
Unidad de Organización Curricular:	Profesional.
Nivel y Paralelo:	Cuarto - "A"
Alumnos participante(s):	Nata Analuiza Walter Alexis
Asignatura:	Metodologías Ágiles
Docente:	Ing. Hernan Naranjo, Mg.

2 Informe de guía práctica

2.1 Objetivos

2.1.1 General

Redactar un objetivo orientado a que los estudiantes comprendan y apliquen el ciclo TDD (Red, Green, Refactor) mediante la implementación progresiva del cálculo del estadístico T-Student.

2.1.2 Específicos

- Traducir las especificaciones matemáticas de la T-Student en casos de prueba automatizados, validando que falle de manera controlada, antes de escribir el código de producción.
- Codificar la lógica algorítmica necesaria para resolver los problemas planteados, asegurando que todas las pruebas unitarias pasen con éxito en el menor tiempo posible.
- Refactorizar el código fuente de desarrollo para su mejor legibilidad, eliminar si se duplica y asegurar una arquitectura limpia, garantizando el calculo del T-Student, permanezca funcional.

2.2 Modalidad

Presencial.

2.3 Tiempo de duración

Presenciales: 4

No presenciales: 0

2.4 Instrucciones

Redactar instrucciones claras y secuenciales para que los estudiantes desarrollen la práctica. Debe incluirse obligatoriamente lo siguiente:

1. Explicar brevemente qué es TDD y cuál es su importancia dentro de las metodologías ágiles.
2. Configurar el entorno de trabajo con C#, .NET SDK y Visual Studio Code.



3. Crear un proyecto de consola o librería en C#.
4. Crear un proyecto de pruebas utilizando un framework como xUnit, NUnit o MSTest.
5. Aplicar el ciclo TDD: primero escribir una prueba que falle, luego implementar el código mínimo para aprobarla y finalmente refactorizar.
6. Programar progresivamente el cálculo del estadístico T-Student.
7. Validar los resultados utilizando diferentes casos de prueba.
8. Documentar todo el proceso de desarrollo, incluyendo capturas de pantalla, explicación de pruebas, errores encontrados, correcciones realizadas y evidencias de ejecución.
9. Presentar el código fuente, el proyecto de pruebas y el informe final de la práctica.

Diseñar, implementar y documentar las pruebas necesarias. Las pruebas deben incluir, como mínimo:

- Prueba para calcular correctamente la media de una muestra.
- Prueba para calcular la varianza.
- Prueba para calcular la desviación estándar.
- Prueba para calcular el estadístico T-Student con datos válidos.
- Prueba para comparar dos muestras independientes, si aplica.
- Prueba para validar muestras vacías.
- Prueba para validar muestras con un solo dato.
- Prueba para validar entradas nulas.
- Prueba para verificar resultados esperados frente a un cálculo manual o una fuente confiable.
- Prueba de refactorización, evidenciando que el cambio interno del código no altera el resultado esperado.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Computadora
- Herramientas Ofimáticas
- Aula virtual.
- Internet

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas



- ☒ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades a Desarrollar

Describir los productos que se espera obtener al finalizar la práctica, tales como:

- Proyecto funcional en C#.
- Proyecto de pruebas unitarias.
- Implementación del cálculo del estadístico T-Student.
- Evidencia de aplicación del ciclo TDD.
- Informe técnico documentado.
- Repositorio o carpeta comprimida con el código fuente.

2.7 Resultados obtenidos

TDD (Test-Driven Development)

También conocido como Desarrollo guiado por pruebas, es una metodología para el desarrollo de Software que invierten en el orden tradicional de programación en el cual se pone mas énfasis en crear unas pruebas automatizadas antes de crear código. Como se menciona en el enfoque tradicional primero se programa y después se prueba, para ver si tiene fallas, en cambio en los TDD no sigue el mismo ciclo, si no se tiene una prueba que valide si existe una falla, no se escribe ni una sola linea de código.[1] [2]

El Ciclo TDD

El desarrollo basado en pruebas tiene un flujo continuo el cual consta de tres fases:

- **RED:** El desarrollador escribe una prueba automatizada, basada en los requisitos.[1]
- **GREEN:** Se escribe el código funcional, solo lo necesario, para que pase la prueba.[1]
- **REFACTOR:** Al finalizar la prueba, se limpia, optimiza y reestructura, para que sea mas legible y mantenible. [1]

Impacto del TDD en el Software

1. **Calidad del Software:** Gracias a los TDD la cantidad de errores que puede existir en un código, se disminuye, tenemos menos bugs en producción y un aumento notable en la cobertura de código. Al captar los errores de manera temprana, hace que solucionarlo sea mucho mas rápido y económico. [3]



2. **Tiempo de Desarrollo:** Su impacto no es tan lineal, se puede acelerar el time-to-marker, porque es menos tiempo en buscar y parchar bugs.
3. **Productividad y dinámica de Equipo:** Mejora la colaboración entre los desarrolladores y testers y eleva la confianza del equipo.[3]

Desafíos y Desventajas

- Requiere un cambio drásticos en la mentalidad, de los desarrolladores lo que es muy difícil.[4]
- Si el código cambia las pruebas también deben actualizarse.
- Existe la posibilidad de escribir código complejo solo para satisfacer la rigidez de las pruebas. [4]
- Para que funcione los equipos requieren entrenamientos especializados y herramientas de infraestructura. [4]

Importancia en las metodologías ágiles

- La integración de TDD en los marcos ágiles, es crucial para la gestión de requisitos que evolucionan rápidamente. [5]
- Los TDD se alinean perfectamente con las practicas ágiles y los flujos de integración continua al proporcionar un diseño robusto que permiten un desarrollo incremental. [5]
- Entrega rápida con estabilidad.[5]
- El TDD mejora la tolerancia a fallos y reduce errores post-despliegue dentro de equipos de software ágiles. [5]

Configuración del entorno

Como primer paso, se creo el repositorio en git hub con el nombre tdd-tstudent-csharp, para evidenciar todo el desarrollo bajo TDD.

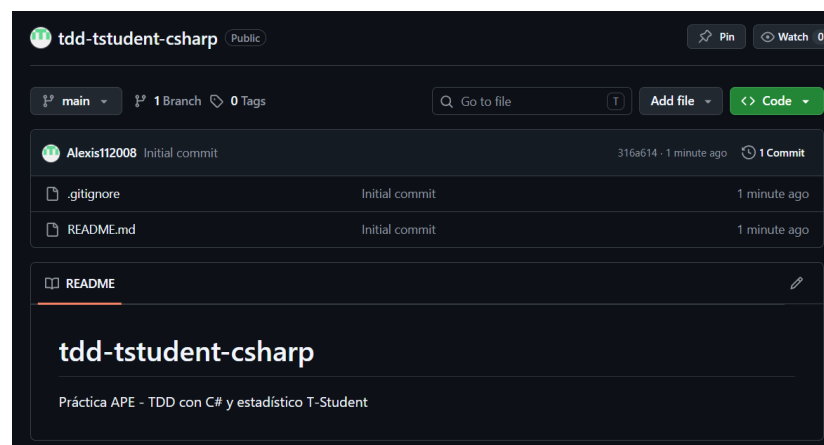


Figure 1: Repositorio creado en GitHub



Se creó la solución de .NET con el comando "dotnet new sln" para agrupar proyectos de librería y de pruebas.

```
LENOVO@DESKTOP-C5HVSNL MINGW64 ~/tdd-tstudent-csharp (main)
$ dotnet new sln -n TStudentTDD
La plantilla "Archivo de la solución" se creó correctamente.

LENOVO@DESKTOP-C5HVSNL MINGW64 ~/tdd-tstudent-csharp (main)
$
```

Figure 2: Creación de la solución con dotnet new sln

Luego creamos un proyecto de tipo classlib llamado TStudentLib, que contiene la logica del calculo del estadístico T-Student.

```
LENOVO@DESKTOP-C5HVSNL MINGW64 ~/tdd-tstudent-csharp (main)
$ dotnet new classlib -n TStudentLib
La plantilla "Biblioteca de clases" se creó correctamente.

Procesando acciones posteriores a la creación...
Restaurando C:\Users\LENOVO\tdd-tstudent-csharp\TStudentLib\TStudentLib.csproj:
Restauración realizada correctamente.

LENOVO@DESKTOP-C5HVSNL MINGW64 ~/tdd-tstudent-csharp (main)
$
```

Figure 3: Proyecto para la logica del calculo

Se creó un proyecto de pruebas con xUnit llamado TStudentTests, donde se implementarán todas las pruebas del ciclo TDD.

```
LENOVO@DESKTOP-C5HVSNL MINGW64 ~/tdd-tstudent-csharp (main)
$ dotnet new xunit -n TStudentTests
La plantilla "Proyecto de prueba de xUnit" se creó correctamente.

Procesando acciones posteriores a la creación...
Restaurando C:\Users\LENOVO\tdd-tstudent-csharp\TStudentTests\TStudentTests.csproj:
Restauración realizada correctamente.

LENOVO@DESKTOP-C5HVSNL MINGW64 ~/tdd-tstudent-csharp (main)
$
```

Figure 4: Proyecto para pruebas del TDD

Se agregaron ambos proyectos a la solución usando dotnet sln add.



```
LENOVO@DESKTOP-C5HVSNL MINGW64 ~/tdd-tstudent-csharp (main)
$ dotnet sln TStudentTDD.slnx add TStudentLib/TStudentLib.csproj
Se ha agregado el proyecto "TStudentLib\TStudentLib.csproj" a la solución.

LENOVO@DESKTOP-C5HVSNL MINGW64 ~/tdd-tstudent-csharp (main)
$ dotnet sln TStudentTDD.slnx add TStudentTests/TStudentTests.csproj
Se ha agregado el proyecto "TStudentTests\TStudentTests.csproj" a la solución.

LENOVO@DESKTOP-C5HVSNL MINGW64 ~/tdd-tstudent-csharp (main)
$
```

Figure 5: Proyecto en la solución

Se añadió la referencia del proyecto de pruebas hacia la librería para que las pruebas puedan acceder a las clases y métodos a probar.

```
LENOVO@DESKTOP-C5HVSNL MINGW64 ~/tdd-tstudent-csharp (main)
$ dotnet add TStudentTests/TStudentTests.csproj reference TStudentLib/TStudentLib.csproj
Se ha agregado la referencia "..\TStudentLib\TStudentLib.csproj" al proyecto.

LENOVO@DESKTOP-C5HVSNL MINGW64 ~/tdd-tstudent-csharp (main)
$
```

Figure 6: Referencias hacia las librerías

Estructura del proyecto

la estructura del directorio se corresponde con aquella que queda organizada de la siguiente manera: La solución TStudentTDD.slnx, librería TStudentLib que contendrá la lógica de negocio, el proyecto de pruebas TStudentTests con xUnit, los archivos de configuración .gitignore y README.md.

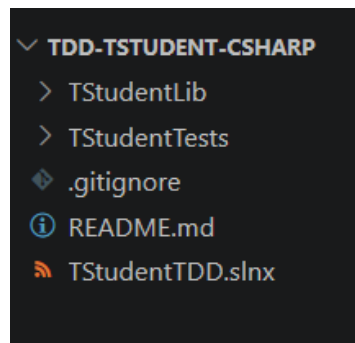


Figure 7: Estructura del proyecto

Aplicación del ciclo TDD

Prueba para calcular correctamente la media de una muestra

RED: Escribir la prueba de la media

En la carpeta TStudentTests/UnitTest.cs vamos a codificar el método CalcularMedia(), donde recibe un arreglo, en donde devuelve un double, si el método no está implementado o devuelve otro valor falla.



```
namespace TStudentTests;

public class TStudentCalculatorTests
{
    private readonly TStudentLib.TStudentCalculator _calc;

    public TStudentCalculatorTests()
    {
        _calc = new TStudentLib.TStudentCalculator();
    }

    // PRUEBA 1: Media
    [Fact]
    public void CalcularMedia_ConDatosValidos_RetornaMediaCorrecta()
    {
        double[] muestra = { 2, 4, 6, 8, 10 };
        double resultadoEsperado = 6.0;

        double resultado = _calc.CalcularMedia(muestra);

        Assert.Equal(resultadoEsperado, resultado);
    }
}
```

Figure 8: Prueba RED calculo de la media

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx" y como observamos falla.

```
PS C:\Users\LENOVO\tdd-tstudent-csharp> dotnet test TStudentTDD.slnx
Restauración completada (0,9s)
TStudentLib net10.0 realizado correctamente (0,2s) → TStudentLib\bin\Debug\net10.0\TStudentLib.dll
TStudentTests net10.0 error con 1 errores (2,7s)
C:\Users\LENOVO\tdd-tstudent-csharp\TStudentTests\UnitTest1.cs(5,34): error CS0234: El tipo o el nombre del espacio
o de nombres 'TStudentCalculator' no existe en el espacio de nombres 'TStudentLib' (¿falta alguna referencia de ensamblado?)

Compilación error con 1 errores en 4,1s
PS C:\Users\LENOVO\tdd-tstudent-csharp>
```

Figure 9: Salida de la prueba RED

GREEN: Implementar CalcularMedia

En el archivo TStudentLib/class1.cs vamos a implementar lo mínimo para que la prueba pase en el cual el método de CalcularMedia(), recibe un arreglo, verifica que no se nulo, después sumamos las cantidades y dividimos entre la cantidad de elementos y obtenemos a la media.



```
namespace TStudentLib;

public class TStudentCalculator
{
    // MÉTODO 1: Calcular Media
    public double CalcularMedia(double[] muestra)
    {
        if (muestra == null || muestra.Length == 0)
            throw new ArgumentException("La muestra no puede ser nula o vacía.");

        double suma = 0;
        foreach (double valor in muestra)
            suma += valor;

        return suma / muestra.Length;
    }
}
```

Figure 10: Implementación método Calcular Media

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx" y como observamos la prueba paso.

```
PS C:\Users\LENOVO\tdd-tstudent-csharp> dotnet test TStudentTDD.slnx
Restauración completada (0,9s)
TStudentLib net10.0 realizado correctamente (2,6s) → TStudentLib\bin\Debug\net10.0
TStudentTests net10.0 realizado correctamente (0,9s) → TStudentTests\bin\Debug\net10.0
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0)
[xUnit.net 00:00:00.41] Discovering: TStudentTests
[xUnit.net 00:00:00.46] Discovered: TStudentTests
[xUnit.net 00:00:00.49] Starting: TStudentTests
[xUnit.net 00:00:00.70] Finished: TStudentTests
TStudentTests net10.0 realizado correctamente prueba (1,9s)

Resumen de pruebas: total: 1; con errores: 0; correcto: 1; omitido: 0; duración: 1,9s
Compilación realizado correctamente en 6,6s
PS C:\Users\LENOVO\tdd-tstudent-csharp>
```

Figure 11: Salida de la prueba GREEN

REFACTOR: Media

En el archivo class1.cs vamos a refactorizar el código, hemos agregado comentarios XML, el cual recibe que recibe el método, que retorna y que hace.



```
namespace TStudentLib;

public class TStudentCalculator
{
    /// <summary>
    /// Calcula la media aritmética de una muestra.
    /// </summary>
    /// <param name="muestra">Arreglo de valores numéricos.</param>
    /// <returns>Media aritmética de la muestra.</returns>
    public double CalcularMedia(double[] muestra)
    {
        if (muestra == null || muestra.Length == 0)
            throw new ArgumentException("La muestra no puede ser nula o vacía.");

        double suma = 0;
        foreach (double valor in muestra)
            suma += valor;

        return suma / muestra.Length;
    }
}
```

Figure 12: Refactorizar código de media

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx" y como observamos que la prueba no se altero los datos siguen pasando.

```
PS C:\Users\LENOVO\tdd-tstudent-csharp> dotnet test TStudentTDD.slnx
Restauración completada (0,9s)
TStudentLib net10.0 realizado correctamente (2,6s) → TStudentLib\bin\Debug\net10.0
TStudentTests net10.0 realizado correctamente (0,9s) → TStudentTests\bin\Debug\net10.0
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0)
[xUnit.net 00:00:00.41] Discovering: TStudentTests
[xUnit.net 00:00:00.46] Discovered: TStudentTests
[xUnit.net 00:00:00.49] Starting: TStudentTests
[xUnit.net 00:00:00.70] Finished: TStudentTests
TStudentTests net10.0 realizado correctamente prueba (1,9s)

Resumen de pruebas: total: 1; con errores: 0; correcto: 1; omitido: 0; duración: 1,9s
Compilación realizado correctamente en 6,6s
PS C:\Users\LENOVO\tdd-tstudent-csharp>
```

Figure 13: Salida de la prueba Refactor

Prueba para calcular la varianza.

RED: Escribir la prueba del calculo de la varianza

En la carpeta TStudentTests/UnitTest.cs vamos a codificar el método CalcularVarianza(), donde recibe un arreglo double, y devuelve un double de igual manera, si el método no esta implementado o devuelve otro valor, la prueba falla.



```
// PRUEBA 2: Varianza
[Fact]
public void CalcularVarianza_ConDatosValidos_RetornaVarianzaCorrecta()
{
    double[] muestra = { 2, 4, 6, 8, 10 };
    double resultadoEsperado = 8.0; // varianza muestral

    double resultado = _calc.CalcularVarianza(muestra);

    Assert.Equal(resultadoEsperado, resultado);
}
```

Figure 14: Prueba RED calculo de la varianza

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx" y como observamos la prueba falla.

```
PS C:\Users\LENOVO\tdd-tstudent-csharp> dotnet test TStudentTDD.slnx
Restauración completada (0,9s)
TStudentLib net10.0 realizado correctamente (2,6s) → TStudentLib\bin\Debug\net10.0\TStudentLib.dll
TStudentTests net10.0 error con 1 errores (0,7s)
C:\Users\LENOVO\tdd-tstudent-csharp\TStudentTests\UnitTest1.cs(3,5): error CS0106: El modificador 'public' no es v
álido para este elemento

Compilación error con 1 errores en 4,4s
PS C:\Users\LENOVO\tdd-tstudent-csharp>
```

Figure 15: Salida de la prueba RED

GREEN: Implementar CalcularVarianza

En el archivo TStudentLib/class1.cs implementaremos el método de CalcularVarianza(), verifica que no sea nulo y tenga dos elementos. Sumando los cuadrados de diferencia entre cada valor y la media usando Math.Pow y por ultimo divide (n-1) para retornar la varianza.

```
public double CalcularVarianza(double[] muestra)
{
    if (muestra == null || muestra.Length < 2)
        throw new ArgumentException("La muestra debe tener al menos 2 elementos.");

    double media = CalcularMedia(muestra);
    double sumaCuadrados = 0;

    foreach (double valor in muestra)
        sumaCuadrados += Math.Pow(valor - media, 2);

    return sumaCuadrados / (muestra.Length - 1);
}
```

Figure 16: Implementación método Calcular Varianza

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx -logger "console;verbosity=detailed"" como vemos en la captura vemos que los dos métodos pasaron correctamente.



```
PS C:\Users\LENOVO\tdtdd-tstudent-csharp> dotnet test TStudentTDD.slnx --logger "console;verbosity=detailed"
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.5)
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.5)
[xUnit.net 00:00:00.09] Discovering: TStudentTests
[xUnit.net 00:00:00.09] Discovering: TStudentTests
[xUnit.net 00:00:00.12] Discovered: TStudentTests
[xUnit.net 00:00:00.12] Discovered: TStudentTests
[xUnit.net 00:00:00.15] Starting: TStudentTests
[xUnit.net 00:00:00.15] Starting: TStudentTests
[xUnit.net 00:00:00.22] Finished: TStudentTests
[xUnit.net 00:00:00.22] Finished: TStudentTests
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_ConDatosValidos_RetornaMediaCorrecta [27 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_ConDatosValidos_RetornaVarianzaCorrecta [< 1 ms]

La serie de pruebas se ejecutó correctamente.
Pruebas totales: 2
Correcto: 2
Tiempo total: 0,7386 Segundos
TStudentTests net10.0 realizado correctamente prueba (1,0s)

Resumen de pruebas: total: 2; con errores: 0; correcto: 2; omitido: 0; duración: 1,0 s
Compilación realizado correctamente en 2,3s
```

Figure 17: Salida de la prueba GREEN

REFACTOR: Varianza

En el archivo TStudentLib/class1.cs vamos a refactorizar el código, hemos agregado comentarios XML, el cual recibe que recibe el método, que retorna y que hace.

```
/// <summary> Calcula la varianza de una muestra de datos.
/// </summary> <param name="muestra">Arreglo de valores numéricos.</param>
/// <returns>Varianza de la muestra.</returns>
public double CalcularVarianza(double[] muestra)
{
    if (muestra == null || muestra.Length < 2)
        throw new ArgumentException("La muestra debe tener al menos 2 elementos.");

    double media = CalcularMedia(muestra);
    double sumaCuadrados = 0;

    foreach (double valor in muestra)
        sumaCuadrados += Math.Pow(valor - media, 2);

    return sumaCuadrados / (muestra.Length - 1);
}
```

Figure 18: Refactorizar código de calcular varianza

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx --logger "console;verbosity=detailed"" y como observamos que la prueba no se altero los datos siguen pasando.



```
PS C:\Users\LENOVO\tdd-tstudent-csharp> dotnet test TStudentTDD.slnx --logger "console;verbosity=detailed"
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.5)
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.5)
[xUnit.net 00:00:00.09] Discovering: TStudentTests
[xUnit.net 00:00:00.09] Discovering: TStudentTests
[xUnit.net 00:00:00.12] Discovered: TStudentTests
[xUnit.net 00:00:00.12] Discovered: TStudentTests
[xUnit.net 00:00:00.15] Starting: TStudentTests
[xUnit.net 00:00:00.15] Starting: TStudentTests
[xUnit.net 00:00:00.22] Finished: TStudentTests
[xUnit.net 00:00:00.22] Finished: TStudentTests
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_ConDatosValidos_RetornaMediaCorrecta [27 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_ConDatosValidos_RetornaVarianzaCorrecta [< 1 ms]

La serie de pruebas se ejecutó correctamente.
Pruebas totales: 2
Correcto: 2
Tiempo total: 0,7386 Segundos
TStudentTests net10.0 realizado correctamente prueba (1,0s)

Resumen de pruebas: total: 2; con errores: 0; correcto: 2; omitido: 0; duración: 1,0 s
Compilación realizado correctamente en 2,3s
```

Figure 19: Salida de la prueba Refactor

Prueba para calcular la desviación estándar

RED: Escribir la prueba de calcular la desviación estándar

En la carpeta TStudentTests/UnitTest.cs implementaremos el método CalcularDesviacionEstandar(), recibe un arreglo y nos retorna una raíz cuadrada de 10.0. Si la implementación del método no coincide entonces falla la prueba.

```
// PRUEBA 3: Desviación Estándar
[Fact]
public void CalcularDesviacionEstandar_ConDatosValidos_RetornaDesviacionCorrecta()
{
    double[] muestra = { 2, 4, 6, 8, 10 };
    double resultadoEsperado = Math.Sqrt(10.0);

    double resultado = _calc.CalcularDesviacionEstandar(muestra);

    Assert.Equal(resultadoEsperado, resultado, 10);
}
```

Figure 20: Prueba RED de calcular la desviacion estándar

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx" y como observamos la prueba falla, nos sale el nombre del método que no paso la prueba.

```
PS C:\Users\LENOVO\tdd-tstudent-csharp> dotnet test TStudentTDD.slnx
Restauración completada (0,6s)
TStudentLib net10.0 realizado correctamente (2,4s) → TStudentLib\bin\Debug\net10.0\TStudentLib.dll
TStudentTests net10.0 error con 1 errores (0,6s)
C:\Users\LENOVO\tdd-tstudent-csharp\TStudentTests\UnitTest1.cs(38,34): error CS1061: "TStudentCalculator" no contiene una definición para "CalcularDesviacionEstandar" ni un método de extensión accesible "CalcularDesviacionEstandar" que acepte un primer argumento del tipo "TStudentCalculator" (¿falta alguna directiva using o una referencia de ensamblado?)
Compilación error con 1 errores en 3,9s
PS C:\Users\LENOVO\tdd-tstudent-csharp>
```

Figure 21: Salida de la prueba RED



GREEN: Implementar CalcularDesviacionEstandar

En el archivo TStudentLib/class1.cs implementaremos el método de CalcularDesviacionEstandar, aplicando la raíz cuadrada a la varianza muestral. No repite lógica de validación.

```
public double CalcularDesviacionEstandar(double[] muestra)
{
    return Math.Sqrt(CalcularVarianza(muestra));
}
```

Figure 22: Implementación método Calcular Desviación Estándar

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx -logger "console;verbosity=detailed"" como vemos en la captura vemos que los tres métodos pasaron correctamente.

```
[xUnit.net 00:00:00.22] Finished: TStudentTests
[xUnit.net 00:00:00.22] Finished: TStudentTests
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_ConDatosValidos_RetornaMediaCorrecta [26 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularDesviacionEstandar_ConDatosValidos_RetornaDesviacionCorrecta
[< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_ConDatosValidos_RetornaVarianzaCorrecta [< 1 ms]

La serie de pruebas se ejecutó correctamente.
Pruebas totales: 3
Correcto: 3
Tiempo total: 0,7242 Segundos
TStudentTests net10.0 realizado correctamente prueba (0,9s)

Resumen de pruebas: total: 3; con errores: 0; correcto: 3; omitido: 0; duración: 0,9 s
Compilación realizado correctamente en 2,3s
PS C:\Users\LENDVO\tdd-tstudent-csharp>
```

Figure 23: Salida de la prueba GREEN

REFACTOR: Desviación Estándar

En el archivo TStudentLib/class1.cs vamos a refactorizar el código, hemos agregado comentarios XML, el cual nos indica que recibe el método, que retorna y que hace.

```
/// <summary>
/// Calcula la desviación estándar muestral de una muestra.
/// </summary> <param name="muestra">Arreglo de valores numéricos.</param>
/// <returns>Desviación estándar de la muestra.</returns>
public double CalcularDesviacionEstandar(double[] muestra)
{
    return Math.Sqrt(CalcularVarianza(muestra));
}
```

Figure 24: Refactorizar código de Desviación Estándar

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx -logger "console;verbosity=detailed"" y como observamos que la prueba no se altero los datos siguen pasando.



```
[xUnit.net 00:00:00.22] Finished: TStudentTests
[xUnit.net 00:00:00.22] Finished: TStudentTests
  Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_ConDatosValidos_RetornaMediaCorrecta [26 ms]
  Correctas TStudentTests.TStudentCalculatorTests.CalcularDesviacionEstandar_ConDatosValidos_RetornaDesviacionCorrecta
  [< 1 ms]
  Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_ConDatosValidos_RetornaVarianzaCorrecta [< 1 ms]

La serie de pruebas se ejecutó correctamente.
Pruebas totales: 3
  Correcto: 3
Tiempo total: 0,7242 Segundos
  TStudentTests net10.0 realizado correctamente prueba (0,9s)

Resumen de pruebas: total: 3; con errores: 0; correcto: 3; omitido: 0; duración: 0,9 s
Compilación realizado correctamente en 2,3s
PS C:\Users\LENOVO\tdd-tstudent-csharp>
```

Figure 25: Salida de la prueba Refactor

Prueba para calcular el estadístico T-Student con datos válidos.

RED: Escribir la prueba de calcular el estadístico T-Student

En la carpeta TStudentTests/UnitTest.cs implementamos la prueba para calcular el estadístico T-Student, comparando la media de la muestra con una media poblacional hipotética de 5.0.

```
// PRUEBA 4: Estadístico T-Student
[Fact]
public void CalcularTStudent_ConDatosValidos_RetornaTStudentCorrecto()
{
    double[] muestra = { 2, 4, 6, 8, 10 };
    double mediaPoblacional = 5.0;
    // t = (media - mu) / (desviacion / sqrt(n))
    // t = (6 - 5) / (sqrt(10) / sqrt(5)) = 1 / (3.162/2.236) = 0.7071
    double resultadoEsperado = 0.7071;

    double resultado = _calc.CalcularTStudent(muestra, mediaPoblacional);

    Assert.Equal(resultadoEsperado, resultado, 4);
}
```

Figure 26: Prueba RED para calcular el estadístico T-Student

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx" y como observamos la prueba falla, nos sale el nombre del método que no paso la prueba.

```
PS C:\Users\LENOVO\tdd-tstudent-csharp> dotnet test TStudentTDD.slnx
Restauración completada (0,9s)
TStudentLib net10.0 realizado correctamente (2,5s) -> TStudentLib\bin\Debug\net10.0\TStudentLib.dll
TStudentTests net10.0 error con 1 errores (0,9s)
  C:\Users\LENOVO\tdd-tstudent-csharp\TStudentTests\UnitTest1.cs(53,34): error CS1061: "TStudentCalculator" no contiene una definición para "CalcularTStudent" ni un método de extensión accesible "CalcularTStudent" que acepte un primer argumento del tipo "TStudentCalculator" (¿falta alguna directiva using o una referencia de ensamblado?)

Compilación error con 1 errores en 4,5s
PS C:\Users\LENOVO\tdd-tstudent-csharp>
```

Figure 27: Salida de la prueba RED

GREEN: Implementar Calcular el estadístico T-Student



En el archivo TStudentLib/class1.cs calcula el estadístico T-Student usando la formula $(\text{media muestral} - \text{media poblacional}) / (\text{desviación estándar} / \sqrt{n})$, primero valida que existan 2 valores y reutiliza los métodos de la media y la desviación para los valores necesarios.

```
public double CalcularTStudent(double[] muestra, double mediaPoblacional)
{
    if (muestra == null || muestra.Length < 2)
        throw new ArgumentException("La muestra debe tener al menos 2 elementos.");

    double media = CalcularMedia(muestra);
    double desviacion = CalcularDesviacionEstandar(muestra);
    double n = muestra.Length;

    return (media - mediaPoblacional) / (desviacion / Math.Sqrt(n));
}
```

Figure 28: Implementación método Calcular Estadístico T-Student

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx -logger "console;verbosity=detailed"" como vemos en la captura vemos que los cuatro métodos pasaron correctamente.

```
[xUnit.net 00:00:00.23] Finished: TStudentTests
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_ConDatosValidos_RetornaMediaCorrecta [27 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularDesviacionEstandar_ConDatosValidos_RetornaDesviacionCorrecta
[< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_ConDatosValidos_RetornaVarianzaCorrecta [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudent_ConDatosValidos_RetornaTStudentCorrecto [< 1 ms]

La serie de pruebas se ejecutó correctamente.
Pruebas totales: 4
Correcto: 4
Tiempo total: 0,7635 Segundos
TStudentTests net10.0 realizado correctamente prueba (1,0s)

Resumen de pruebas: total: 4; con errores: 0; correcto: 4; omitido: 0; duración: 1,0 s
Compilación realizado correctamente en 2,4s
PS C:\Users\LENOVO\tdd-tstudent-csharp> |
```

Figure 29: Salida de la prueba GREEN

REFACTOR: Calculo de Estadístico T-Student

En el archivo TStudentLib/class1.cs vamos a refactorizar el código, hemos agregado comentarios XML, el cual nos indica que recibe el método, que retorna y que hace.

```
/// <summary> Calcula el estadístico T-Student para una muestra dada una media poblacional
/// </summary> <param name="muestra">Arreglo de valores numéricos.</param> <param name="me
/// <returns>Valor del estadístico T-Student.</returns>
public double CalcularTStudent(double[] muestra, double mediaPoblacional)
{
    if (muestra == null || muestra.Length < 2)
        throw new ArgumentException("La muestra debe tener al menos 2 elementos.");

    double media = CalcularMedia(muestra);
    double desviacion = CalcularDesviacionEstandar(muestra);
    double n = muestra.Length;

    return (media - mediaPoblacional) / (desviacion / Math.Sqrt(n));
}
```

Figure 30: Refactorizar código para calcular el estadístico T-Student



Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx -logger "console;verbosity=detailed"" y como observamos que la prueba no se altero los datos siguen pasando las pruebas.

```
[xUnit.net 00:00:00.23] Finished: TStudentTests
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_ConDatosValidos_RetornaMediaCorrecta [27 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularDesviacionEstandar_ConDatosValidos_RetornaDesviacionCorrecta
[< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_ConDatosValidos_RetornaVarianzaCorrecta [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudent_ConDatosValidos_RetornaTStudentCorrecto [< 1 ms]

La serie de pruebas se ejecutó correctamente.
Pruebas totales: 4
Correcto: 4
Tiempo total: 0,7635 Segundos
TStudentTests net10.0 realizado correctamente prueba (1,0s)

Resumen de pruebas: total: 4; con errores: 0; correcto: 4; omitido: 0; duración: 1,0 s
Compilación realizado correctamente en 2,4s
PS C:\Users\LENOVO\tdd-tstudent-csharp>
```

Figure 31: Salida de la prueba Refactor

Prueba para comparar dos muestras independientes, si aplica.

RED: Escribir la prueba de calcular el estadístico T-Student para dos muestras independientes

En la carpeta TStudentTests/UnitTest.cs implementamos la prueba para calcular el estadístico T-Student de dos muestras. Aplica la fórmula $(media1 - media2) / (\text{var1}/n1 + \text{var2}/n2)$, esperando un valor de 0.5 con tolerancia de 4 decimales.

```
// PRUEBA 5: Dos muestras independientes
[Fact]
public void CalcularTStudentDosMuestras_ConDatosValidos_RetornaTStudentCorrecto()
{
    double[] muestra1 = { 2, 4, 6, 8, 10 };
    double[] muestra2 = { 1, 3, 5, 7, 9 };
    // t = (media1 - media2) / sqrt(var1/n1 + var2/n2)
    // t = (6 - 5) / sqrt(10/5 + 10/5) = 1 / sqrt(4) = 0.5
    double resultadoEsperado = 0.5;

    double resultado = _calc.CalcularTStudentDosMuestras(muestra1, muestra2);

    Assert.Equal(resultadoEsperado, resultado, 4);
}
```

Figure 32: Prueba RED para calcular el estadístico T-Student de dos muestras

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx" y como observamos la prueba falla, nos sale el nombre del método que no paso la prueba.



```
PS C:\Users\LENOVO\tdd-tstudent-csharp> dotnet test TStudentTDD.slnx
Restauración completada (0,9s)
TStudentLib net10.0 realizado correctamente (2,6s) -> TStudentLib\bin\Debug\net10.0\TStudentLib.dll
TStudentTests net10.0 error con 1 errores (0,9s)
C:\Users\LENOVO\tdd-tstudent-csharp\TStudentTests\UnitTest1.cs(68,34): error CS1061: "TStudentCalculator" no contiene una definición para "CalcularTStudentDosMuestras" ni un método de extensión accesible "CalcularTStudentDosMuestras" que acepte un primer argumento del tipo "TStudentCalculator" (¿falta alguna directiva using o una referencia de ensamblado?)
Compilación error con 1 errores en 4,6s
PS C:\Users\LENOVO\tdd-tstudent-csharp>
```

Figure 33: Salida de la prueba RED

GREEN: Implementar Calcular el estadístico T-Student para dos muestras

En el archivo TStudentLib/class1.cs calcula el estadístico T-Student para dos valores independientes. Calcula la diferencia entre sus medidas y divide por el error estándar, al combinarlos se obtiene el estadístico T que indica la diferencia que tiene entre ellos.

```
public double CalcularTStudentDosMuestras(double[] muestra1, double[] muestra2)
{
    if (muestra1 == null || muestra1.Length < 2)
        throw new ArgumentException("La muestra 1 debe tener al menos 2 elementos.");
    if (muestra2 == null || muestra2.Length < 2)
        throw new ArgumentException("La muestra 2 debe tener al menos 2 elementos.");

    double media1 = CalcularMedia(muestra1);
    double media2 = CalcularMedia(muestra2);
    double var1 = CalcularVarianza(muestra1);
    double var2 = CalcularVarianza(muestra2);
    double errorEstandar = Math.Sqrt(var1 / muestra1.Length + var2 / muestra2.Length);

    return (media1 - media2) / errorEstandar;
}
```

Figure 34: Implementación método Calcular Estadístico T-Student en dos muestras

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx -logger "console;verbosity=detailed"" como vemos en la captura vemos que los cinco métodos pasaron correctamente.

```
[xUnit.net 00:00:00.23] Finished: TStudentTests
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_ConDatosValidos_RetornaMediaCorrecta [29 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularDesviacionEstandar_ConDatosValidos_RetornaDesviacionCorrecta
[< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_ConDatosValidos_RetornaVarianzaCorrecta [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudent_ConDatosValidos_RetornaTStudentCorrecto [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudentDosMuestras_ConDatosValidos_RetornaTStudentCorrecto
[< 1 ms]

La serie de pruebas se ejecutó correctamente.
Pruebas totales: 5
Correcto: 5
Tiempo total: 0,7692 Segundos
TStudentTests net10.0 realizado correctamente prueba (1,0s)

Resumen de pruebas: total: 5; con errores: 0; correcto: 5; omitido: 0; duración: 1,0 s
Compilación realizado correctamente en 2,4s
PS C:\Users\LENOVO\tdd-tstudent-csharp>
```

Figure 35: Salida de la prueba GREEN

REFACTOR: Calcular Estadístico T-Student para dos muestras independientes



En el archivo TStudentLib/class1.cs vamos a refactorizar el código, hemos agregado comentarios XML, el cual nos indica que recibe el método, que retorna y que hace.

```
/// <summary> Calcula el estadístico T-Student para dos muestras independientes.  
/// </summary> <param name="muestra1">Arreglo de valores numéricos de la primera muestra.</param>  
/// <param name="muestra2">Arreglo de valores numéricos de la segunda muestra.</param>  
/// <returns>Valor del estadístico T-Student.</returns>  
public double CalcularTStudentDosMuestras(double[] muestra1, double[] muestra2)  
{  
    if (muestra1 == null || muestra1.Length < 2)  
        throw new ArgumentException("La muestra 1 debe tener al menos 2 elementos.");  
    if (muestra2 == null || muestra2.Length < 2)  
        throw new ArgumentException("La muestra 2 debe tener al menos 2 elementos.");  
  
    double media1 = CalcularMedia(muestra1);  
    double media2 = CalcularMedia(muestra2);  
    double var1 = CalcularVarianza(muestra1);  
    double var2 = CalcularVarianza(muestra2);  
    double errorEstandar = Math.Sqrt(var1 / muestra1.Length + var2 / muestra2.Length);  
  
    return (media1 - media2) / errorEstandar;  
}
```

Figure 36: Refactorizar código para calcular el estadístico T-Student para dos muestras

Después en nuestra terminal vamos a ejecutar el código con el siguiente comando "dotnet test TStudentTDD.slnx -logger "console;verbosity=detailed"" y como observamos que la prueba no se altero los datos siguen pasando las pruebas.

```
[xUnit.net 00:00:00.23] Finished: TStudentTests  
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_ConDatosValidos_RetornaMediaCorrecta [29 ms]  
Correctas TStudentTests.TStudentCalculatorTests.CalcularDesviacionEstandar_ConDatosValidos_RetornaDesviacionCorrecta  
[< 1 ms]  
Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_ConDatosValidos_RetornaVarianzaCorrecta [< 1 ms]  
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudent_ConDatosValidos_RetornaTStudentCorrecto [< 1 ms]  
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudentDosMuestras_ConDatosValidos_RetornaTStudentCorrecto  
[< 1 ms]  
  
La serie de pruebas se ejecutó correctamente.  
Pruebas totales: 5  
Correcto: 5  
Tiempo total: 0,7692 Segundos  
TStudentTests net10.0 realizado correctamente prueba (1,0s)  
  
Resumen de pruebas: total: 5; con errores: 0; correcto: 5; omitido: 0; duración: 1,0 s  
Compilación realizado correctamente en 2,4s  
PS C:\Users\LENOVO\tdd-tstudent-csharp>
```

Figure 37: Salida de la prueba Refactor

Prueba para validar muestras vacías.

En esta caso implementamos las 3 pruebas siguientes:

- Prueba para validar muestras vacías.
- Prueba para validar muestras con un solo dato.
- Prueba para validar entradas nulas.

RED: Escribir la prueba para validar muestras vacías



En la carpeta TStudentTests/UnitTest.cs Al incluir estas tres pruebas, hace que el sistema maneje de la mejor manera situaciones invalidas o un limite de muestras vacías. Si es el caso, se espera que el método lance una excepción ArgumentException, en lugar de hacer cálculos sin sentido.

```
// PRUEBA 6: Muestra vacía
[Fact]
public void CalcularMedia_MuestraVacía_LanzaExcepcion()
{
    double[] muestra = { };

    Assert.Throws<ArgumentException>(() => _calc.CalcularMedia(muestra));
}

// PRUEBA 7: Muestra con un solo dato
[Fact]
public void CalcularVarianza_UnSoloDato_LanzaExcepcion()
{
    double[] muestra = { 5 };

    Assert.Throws<ArgumentException>(() => _calc.CalcularVarianza(muestra));
}

// PRUEBA 8: Entrada nula
[Fact]
public void CalcularMedia_EntradaNula_LanzaExcepcion()
{
    double[]? muestra = null;

    Assert.Throws<ArgumentException>(() => _calc.CalcularMedia(muestra!));
}
```

Figure 38: Prueba RED para validar muestras vacías

GREEN: Muestras vacías

En este caso, debemos dirigirnos a la fase de GREEN, porque ya tenemos validaciones implementadas en el caso anterior. Al ejecutar este comando "dotnet test TStudentTDD.slnx -logger "console;verbosity=detailed"", y debemos observar los 8 pruebas pasadas.

```
[xUnit.net 00:00:00.22] Finished: TStudentTests
Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_UnSoloDato_LanzaExcepcion [27 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_EntradaNula_LanzaExcepcion [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_ConDatosValidos_RetornaMediaCorrecta [2 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularDesviacionEstandar_ConDatosValidos_RetornaDesviacionCorrecta
[< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_MuestraVacía_LanzaExcepcion [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_ConDatosValidos_RetornaVarianzaCorrecta [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudent_ConDatosValidos_RetornaTStudentCorrecto [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudentDosMuestras_ConDatosValidos_RetornaTStudentCorrecto
[< 1 ms]

La serie de pruebas se ejecutó correctamente.
Pruebas totales: 8
Correcto: 8
Tiempo total: 0,7397 Segundos
TStudentTests net10.0 realizado correctamente prueba (1,0s)

Resumen de pruebas: total: 8; con errores: 0; correcto: 8; omitido: 0; duración: 0,9 s
Compilación realizado correctamente en 5,0s
%PS C:\Users\LENOVO\tdd-tstudent-csharp> dotnet test TStudentTDD.slnx --logger "console;verbosity=detailed"
```

Figure 39: Salida prueba GREEN



REFACTOR: Muestras vacías

En este caso no se requieren cambios estructurales. El código ya maneja correctamente los casos de borde gracias a las validaciones implementadas en los ciclos anteriores. Gracias a esto podemos demostrar que un buen diseño inicial, reduce la necesidad de refactorización.

Prueba para verificar resultados esperados frente a un cálculo manual o una fuente confiable.

RED: Escribir la prueba para verificar resultado ante una prueba manual

En la carpeta TStudentTests/UnitTest.cs. En este caso esta prueba sirve para comparar, cálculos que nosotros resolvimos en una hoja a papel o en cualquier otro método de la resolución de un problema.

```
// PRUEBA 9: Verificar contra cálculo manual
[Fact]
public void CalcularTStudent_VerificadoManualmente_RetornaValorEsperado()
{
    // Cálculo manual verificado:
    // Muestra: {5, 7, 9, 11, 13}
    // Media muestral = 9
    // Varianza = 10
    // Desviación estandar = 3.1623
    // Media poblacional = 7
    // t = (9 - 7) / (3.1623 / sqrt(5)) = 2 / 1.4142 = 1.4142
    double[] muestra = { 5, 7, 9, 11, 13 };
    double mediaPoblacional = 7.0;
    double resultadoEsperado = 1.4142;

    double resultado = _calc.CalcularTStudent(muestra, mediaPoblacional);

    Assert.Equal(resultadoEsperado, resultado, 4);
}
```

Figure 40: Prueba RED para validar resultados

En esta imagen también, podemos observar el cálculo que se realizó, y comparar los resultados:

```
Muestra: {5, 7, 9, 11, 13}
Media muestral = (5+7+9+11+13)/5 = 45/5 = 9
Varianza = [(5-9)2+(7-9)2+(9-9)2+(11-9)2+(13-9)2] / (5-1)
          = [16+4+0+4+16] / 4 = 40/4 = 10
Desviación estándar = √10 = 3.1623
Media poblacional = 7
t = (9 - 7) / (3.1623 / √5) = 2 / 1.4142 = 1.4142
```

Figure 41: Cálculo realizado manualmente

GREEN: Verificación con resultados manuales

En este caso, debemos dirigirnos a la fase de GREEN, porque el método CalcularTStudent ya está implementado desde la prueba 4, y el valor 1.4142 son iguales con lo que el sistema calcula.



Al ejecutar este comando "dotnet test TStudentTDD.slnx -logger "console;verbosity=detailed"", y debemos observar que la prueba paso con esto también verificamos la precisión del algoritmo.

```
PS C:\Users\LENOVO\tdd-tstudent-csharp> dotnet test TStudentTDD.slnx
Restauración completada (0,9s)
TStudentLib net10.0 realizado correctamente (2,6s) -> TStudentLib\bin\Debug\net10.0\TStudentLib.dll
TStudentTests net10.0 realizado correctamente (1,0s) -> TStudentTests\bin\Debug\net10.0\TStudentTests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.5)
[xUnit.net 00:00:00.09] Discovering: TStudentTests
[xUnit.net 00:00:00.13] Discovered: TStudentTests
[xUnit.net 00:00:00.15] Starting: TStudentTests
[xUnit.net 00:00:00.22] Finished: TStudentTests
TStudentTests net10.0 realizado correctamente prueba (1,0s)

Resumen de pruebas: total: 9; con errores: 0; correcto: 9; omitido: 0; duración: 1,0 s
Compilación realizado correctamente en 5,7s
PS C:\Users\LENOVO\tdd-tstudent-csharp>
```

Figure 42: Salida prueba GREEN

REFACTOR: Verificación con cálculos manuales

Hemos aplicado el refactor en la misma prueba, para mejorar la legibilidad de la prueba misma, en donde hemos separado claramente los datos del cálculo esperado con los comentarios más estructurados.

```
// PRUEBA 9: Verificar contra cálculo manual
[Fact]
public void CalcularTStudent_VerificadoManualmente_ReturnaValorEsperado()
{
    // ARRANGE - datos verificados manualmente
    double[] muestra = { 5, 7, 9, 11, 13 };
    double mediaPoblacional = 7.0;

    // Verificación manual:
    // Media muestral = (5+7+9+11+13)/5 = 9
    // Varianza = 10
    // Desv. estándar = sqrt(10) = 3.1623
    // Error estándar = 3.1623 / sqrt(5) = 1.4142
    // t = (9 - 7) / 1.4142 = 1.4142
    double resultadoEsperado = 1.4142;

    // ACT
    double resultado = _calc.CalcularTStudent(muestra, mediaPoblacional);

    // ASSERT
    Assert.Equal(resultadoEsperado, resultado, 4);
}
```

Figure 43: Refactor validaciones

Observamos que estos cambios no afectan al funcionamiento y la prueba nuevamente paso con éxito como lo observamos en la imagen.



```
[xunit.net 00:00:00.21] Finished: TStudentTests
Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_UnSoloDato_LanzaExcepcion [5 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_EntradaNula_LanzaExcepcion [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_ConDatosValidos_RetornaMediaCorrecta [3 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularDesviacionEstandar_ConDatosValidos_RetornaDesviacionCorrecta
[< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_MuestraVacua_LanzaExcepcion [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_ConDatosValidos_RetornaVarianzaCorrecta [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudent_ConDatosValidos_RetornaTStudentCorrecto [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudent_VerificadoManualmente_RetornaValorEsperado [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudentDosMuestras_ConDatosValidos_RetornaTStudentCorrecto
[< 1 ms]

La serie de pruebas se ejecutó correctamente.
Pruebas totales: 9
Correcto: 9
Tiempo total: 0,7253 Segundos
TStudentTests net10.0 realizado correctamente prueba (0,9s)

Resumen de pruebas: total: 9; con errores: 0; correcto: 9; omitido: 0; duración: 0,9 s
Compilación realizado correctamente en 2,2s
PS C:\Users\LENOVO\tdt-tstudent-csharp>
```

Figure 44: Éxito de la prueba

Refactorización

Prueba de refactorización, evidenciando que el cambio interno del código no altera el resultado esperado.

RED: Escribir la prueba para la refactorización

Esta prueba verifica que después de hacer cambios internos al código como limpiar, ordenar o renombrar variables, todos los resultados sigan siendo los mismos de antes. Ejecuta todos los métodos principales como lo son media, varianza, desviación, T-Student, con los mismos datos de siempre y comprueba que sigan dando los valores esperados.

```
// PRUEBA 10: Refactorización - el cambio interno no altera resultados
[Fact]
public void Refactorizacion_CambioInterno_NoAlteraResultados()
{
    // ARRANGE
    double[] muestra = { 2, 4, 6, 8, 10 };
    double mediaPoblacional = 5.0;

    // ACT - ejecutamos todos los métodos para verificar consistencia
    double media = _calc.CalcularMedia(muestra);
    double varianza = _calc.CalcularVarianza(muestra);
    double desviacion = _calc.CalcularDesviacionEstandar(muestra);
    double tStudent = _calc.CalcularTStudent(muestra, mediaPoblacional);

    // ASSERT - verificamos que todos los resultados sean consistentes
    Assert.Equal(6.0, media);
    Assert.Equal(10.0, varianza);
    Assert.Equal(Math.Sqrt(10.0), desviacion);
    Assert.Equal(0.7071, tStudent, 4);
}
```

Figure 45: Prueba RED para refactorización

GREEN: Refactorización



La prueba de refactorización tiene como objetivo verificar que el código interno puede cambiar sin alterar los resultados. Como todos los métodos, CalcularMedia, CalcularVarianza, CalcularDesviacionEstandar, CalcularTStudent, ya fueron implementados y refactorizados en ciclos anteriores, la prueba 10 pasó directamente a GREEN.

```
[xunit.net 00:00:00.15] Starting: TStudentTests
Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_UnSoloDato_LanzaExcepcion [26 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_EntradaNula_LanzaExcepcion [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_ConDatosValidos_RetornaMediaCorrecta [2 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularDesviacionEstandar_ConDatosValidos_RetornaDesviacionCorrecta
[< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.Refactorizacion_CambioInterno_NoAlteraResultados [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularMedia_MuestraVacua_LanzaExcepcion [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularVarianza_ConDatosValidos_RetornaVarianzaCorrecta [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudent_ConDatosValidos_RetornaTStudentCorrecto [< 1 ms]
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudent_VerificadoManualmente_RetornaValorEsperado [< 1 ms]
[xunit.net 00:00:00.25] Finished: TStudentTests
[xunit.net 00:00:00.25] Finished: TStudentTests
Correctas TStudentTests.TStudentCalculatorTests.CalcularTStudentDosMuestras_ConDatosValidos_RetornaTStudentCorrecto
[< 1 ms]

La serie de pruebas se ejecutó correctamente.
Pruebas totales: 10
Correcto: 10
Tiempo total: 0,7409 Segundos
TStudentTests net10.0 realizado correctamente prueba (1,0s)

Resumen de pruebas: total: 10; con errores: 0; correcto: 10; omitido: 0; duración: 1,0 s
compilación realizado correctamente en 2,5s
%PS C:\Users\LENOVO\tdt-tstudent-csharp>
```

Figure 46: Salida prueba GREEN

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- El aplicar el ciclo TDD (Red, Green, Refactor) permitió ir desarrollando el cálculo del estadístico T-Student de forma gradual y controlada para garantizar que cada método a implementar, fuese cumpliendo con los requisitos anteriores a cada implementación ulterior.
- Las pruebas unitarias utilizando xUnit, las cuales sirvieron como una herramienta fundamental para encontrar de forma anticipada errores, como el caso del valor que se encontró erróneo en la varianza muestral, que se subsanó gracias a la etapa RED del ciclo TDD.



- La refactorización del código a través de LINQ, así como del uso de comentarios XML, favorecieron tanto la legibilidad del código aislado como la mantenibilidad del código backend, sin modificar los resultados, respectivamente, confirma que un buen diseño inicial tiene reducción de deuda técnica.
- La validación de los resultados contra los cálculos manuales confirmó la validez del algoritmo implementado, mostrando que el T-Student calculado por el sistema es el mismo que el teórico.
- La integración de TDD con GitHub generó un historial del desarrollo muy claro: cada commit representa una parte del ciclo y aporta trazabilidad acerca del proceso.

2.10 Recomendaciones

- Se sugiere que para todas las pruebas unitarias se ejerza el patrón AAA (Arrange, Act, Assert) porque mejora la legibilidad y entendimiento del fin de la prueba.
- Es fundamental que las pruebas se mantengan actualizadas en el momento en el que cambia el código de producción porque pruebas antiguas pueden dar una falsa sensación de seguridad en el sistema.
- Se aconseja aumentar la realización de pruebas, incluyendo casos con valores negativos, decimales o casos con gran tamaño como muestra para validar la robustez del algoritmo.
- Para proyectos más extensos, se sugiere incluir herramientas de código de cobertura como Coverlet –ya incluido en el proyecto– para saber el porcentaje de código que se cubre mediante las pruebas.
- Se aconseja continuar practicando TDD en proyectos futuros, pues la curva de aprendizaje de inicio se compensará con una reducción de errores en producción y una mayor confianza en el código que se pueda llegar a desarrollar.

2.11 Referencias

Insertar las referencias bibliográficas empleadas aplicando la norma IEEE.

- [1] S. Mylsamy and A. Jain, “Test-driven development (TDD) and code quality,” *Universal Research Reports*, vol. 12, no. 1, pp. 447–454, 2025.
- [2] D. Agha, R. Sohail, A. F. Meghji, R. Qaboolio, and S. Bhatti, “Test driven development and its impact on program design and software quality: A systematic literature review,” *VAWKUM Transactions on Computer Sciences*, vol. 11, no. 1, pp. 268–280, 2023.
- [3] A. Santos, S. Vegas, O. Dieste, F. Uyaguari, A. Tosun, D. Fucci, B. Turhan, G. Scanniello, S. Romano, I. Karac, M. Kuhrmann, V. Mandić, R. Ramač, D. Pfahl, C. Engblom, J. Kyykka, K. Rungi, C. Palomeque, J. Spisak, M. Oivo, and N. Juristo, “A family of experiments on test-driven development,” *Empirical Software Engineering*, vol. 26, pp. 1–53, 2021. arXiv:2011.11942v1.
- [4] S. Vimalleshwaran, T. Sivalingam, J. J. J. Dias, N. Inshaf, D. I. De Silva, and W. M. M. S. Weerasooriya, “Evaluating the impact of test-driven development (TDD) on software,” *International Journal of Science and Engineering Applications*, vol. 12, no. 05, pp. 87–92, 2023.
- [5] J. Cui, “A comparative study on the impact of test-driven development (TDD) and behavior-driven development (BDD) on enterprise software delivery effectiveness,” *IEEE Conference Proceedings*, 2024. Solbridge International School of Business, Daejeon, Republic of Korea.



2.12 Anexos

Se incluye el link del repositorio en el cual se evidencia el trabajo que se siguió para alcanzar el objetivo de la práctica, el repositorio tiene de nombre `tdd-tstudent-csharp`:

<https://github.com/Alexis112008/tdd-tstudent-csharp>